Reference Architecture Brief: Service-level Observability

7 January 2026 - ID G00842494 - 34 min read

By: D.B. Cummings, Dylan Roberts

Initiatives: IT Operations and Cloud Management for Technical Professionals; Evolve Service Management and Cloud Operations

Service-level observability attributes telemetry to service boundaries and leverages signals across the full stack to measure and anticipate health. IT operations specialists should utilize this architectural approach to enhance end-to-end service observability and inform operational decisions.

Architecture Brief

Service-level observability is the capability to assess and interpret a service's health, performance, and reliability in real time using signals that matter to users and the business. It focuses on whether the service meets its defined targets for availability, latency, and error rates.

What is a service?

- **Technical definition:** A service is an independently deployable software component that exposes a discrete function or set of functions through an interface (such as an API or endpoint). It performs a specific task within a system, runs on infrastructure, consumes resources, and often depends on other services. Modern services are typically loosely coupled and designed for interoperability across distributed environments.
- **Logical definition:** A service represents a business capability or operational unit of value. It connects technical execution to business outcomes by enabling measurable impact. For example, a "check-out service" facilitates purchase completion, directly influencing revenue, while an "authentication service" secures user access, supporting compliance and trust.

Service-level observability aggregates and interprets multiple types of signals:

This research note is restricted to the personal use of .

- **Logs:** Time-stamped records of discrete events or messages for diagnostics and auditing.
- **Metrics:** Quantitative measurements of performance, capacity, or health over time.
- **Traces:** End-to-end journeys of requests across components, showing workflows and dependencies.
- **Events:** Structured notifications of significant state changes, such as deployments or failures.
- **Profiles:** Runtime snapshots of resource usage patterns (CPU, memory, I/O) for optimization.

Together, these signals provide a holistic view of service behavior and answer the critical question: *Is the service operating within the parameters that matter most to customers and the business?*

Architecture Use Cases

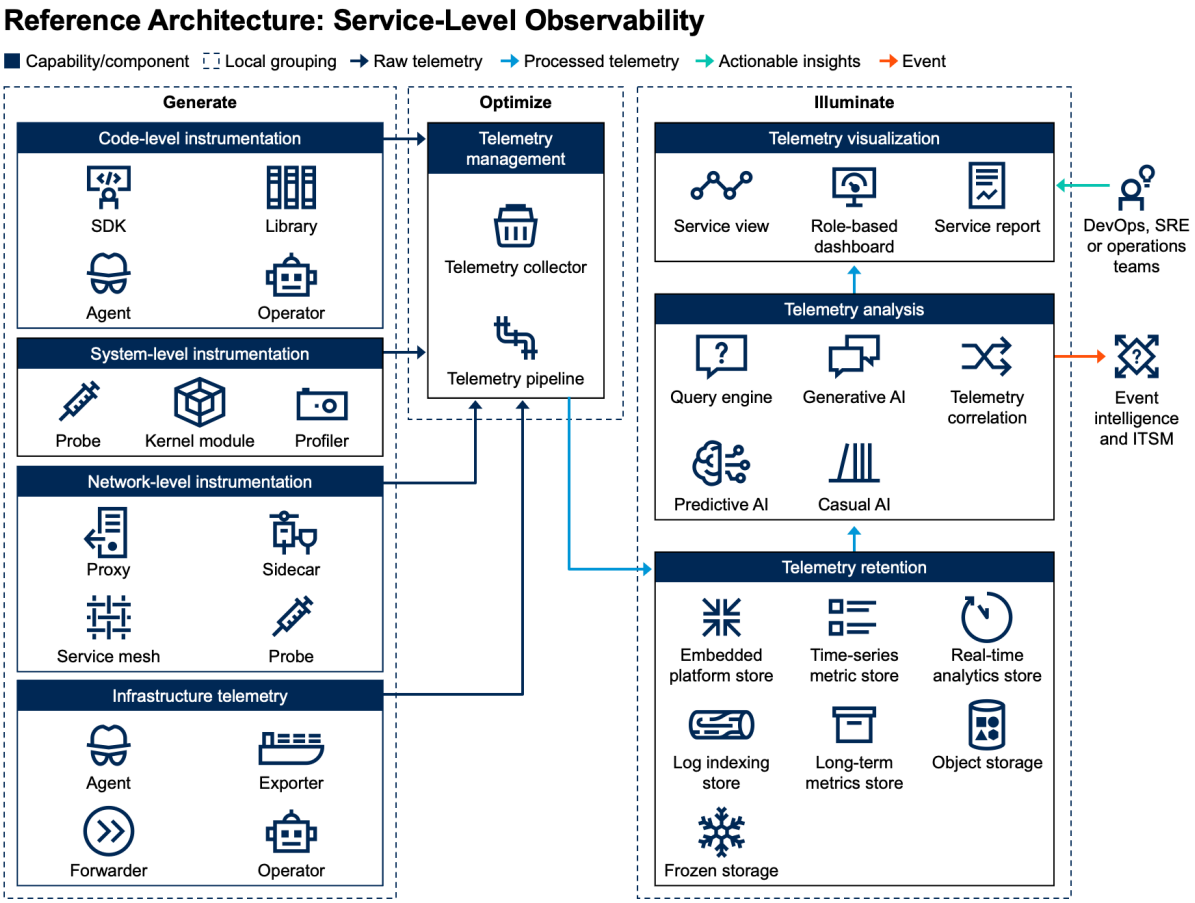
Service-level observability enables organizations to address reliability challenges across modern and traditional architectures. The following use cases benefit most from this architectural approach:

- **Troubleshoot modern service architectures:** Resolve issues across microservices, containers, serverless functions, and service mesh environments by correlating telemetry across layers to identify bottlenecks and root causes.
- **Strengthen developer feedback loops:** Provide visibility into the impact of code changes during testing and deployment, enabling early detection of anomalies and informed release decisions.
- **Enable progressive delivery strategies:** Support canary releases, feature flags, and blue-green deployments by monitoring service health and user experience during incremental rollouts.
- **Monitor and optimize AI-powered services:** Track inference latency, throughput, and model behavior for AI and LLM-based applications, correlating metrics with infrastructure and user experience data.
- **Calculate service reliability and business metrics:** Measure SLAs, transaction success rates, and user experience scores using service-level telemetry to align reliability targets with actual service behavior.

This research note is restricted to the personal use of .

Architecture Diagram

Figure 1: Reference Architecture: Service-Level Observability



The architecture diagram in Figure 1 illustrates the critical elements required for service-level observability and organizes them into three logical groupings, each enclosed by a dashed boundary:

- **Generate:** Instrument services and infrastructure to emit telemetry — logs, metrics, traces, events, and profiles — anchored to defined service boundaries for holistic visibility.
- **Optimize:** Collect, reduce, transform, enrich, and route telemetry using appropriate processing components to ensure service attribution, correlation readiness, and cost-efficient observability across environments.

This research note is restricted to the personal use of .

- **Illuminate:** Surface actionable insights from processed telemetry through correlation, analytics, and visualization — revealing service health, dependencies, and business impact.

Data Flows The diagram also depicts how telemetry moves through the architecture:

- The flow between Generate and Optimize represents raw telemetry being ingested from sources.
- The flow between Optimize and Illuminate represents processed telemetry as it moves toward analysis and visualization through retention.
- Processing may occur at multiple points along the path, beginning with telemetry collectors and continuing through the telemetry pipeline and observability platform components.

Foundational Capabilities The blue-capped boxes in the diagram represent the core capabilities required for service-level observability:

- Instrumentation of software and systems to generate telemetry from services and supporting infrastructure.
- Visibility across service and infrastructure layers to produce correlatable telemetry for holistic analysis.
- Collection, processing, and routing of telemetry data to optimize usability, governance, and cost.
- Retention of telemetry data across tiers for efficient access, compliance, and historical analysis.
- Transformation of telemetry into actionable insight through correlation, analytics, visualization, and automation, illuminating service health, dependencies, and business impact.

Endpoints Two endpoints illustrate how observability outputs are consumed:

- **Human-facing:** DevOps, SRE, and operations teams interact with telemetry for actionable insight in diagnostics, performance optimization, and reliability management.

This research note is restricted to the personal use of .

- **System-facing:** Event Intelligence Solutions and ITSM platforms receive events for automated workflows and incident response.

Architecture Capabilities and Components

The following capabilities represent the core building blocks of this architecture:

- Code-Level Instrumentation
- System-Level Instrumentation
- Network-Level Instrumentation
- Infrastructure Telemetry
- Telemetry Management
- Telemetry Retention
- Telemetry Analysis
- Telemetry Visualization

Code-Level Instrumentation

Back to top

Instrumented software emits telemetry that reflects the state, performance, and health of an application. Telemetry generation occurs within the application code or runtime environment, using technologies that are either manually embedded during development or automatically injected at runtime. Common approaches include:

- **SDKs:** Provide APIs for manual instrumentation during development.
- **Instrumentation libraries:** Native or supplemental libraries that generate telemetry within application code.
- **Telemetry agents:** Automatically inject telemetry logic at runtime, often without modifying source code. Implementation varies by language and platform.
- **Operators:** Kubernetes-native controllers that manage observability components and can automate instrumentation of containerized workloads using custom resources.

This research note is restricted to the personal use of .

Both open-source frameworks (e.g., OpenTelemetry) and proprietary tools support these approaches. Autoinstrumentation typically uses language-specific libraries or telemetry agents to insert telemetry logic automatically, reducing developer effort and enabling consistent observability across services. Telemetry agent-based autoinstrumentation often requires no source code changes and uses runtime injection or binary wrapping to generate telemetry without developer intervention.

Example Technologies

The following are example technologies that enable code-level instrumentation:

- **SDKs:** Dynatrace OneAgent SDK, New Relic Telemetry SDK, OpenTelemetry (OTel) API and SDK
- **Instrumentation Libraries:** OTel instrumentation libraries, Micrometer (Spring Observability), Prometheus client libraries
- **Telemetry Agents:** Datadog Agent, Dynatrace OneAgent, New Relic Agent
- **Operators:** OTel Operator for Kubernetes

This research note is restricted to the personal use of .

Key Characteristics

- Code-level instrumentation alone does not deliver service-level observability. Telemetry must be correlated across services, enriched with context, and analyzed through an observability platform to reveal dependencies, workflows, and root causes.
- Telemetry collection is accelerated through autoinstrumentation and orchestration mechanisms such as telemetry agents and operators. In this context, telemetry agents are software components that run alongside applications to inject telemetry logic at runtime and are not autonomous or decision-making AI systems. Telemetry agents and operators provide an expedient path for generating service-level telemetry without manual code changes.
- Instrumentation libraries enhance granularity for specific frameworks or protocols, complementing autoinstrumentation for deeper visibility into service interactions.
- Manual instrumentation should be minimized to reduce development overhead but leveraged when high-fidelity telemetry or custom service workflows require explicit spans or metrics beyond what autoinstrumentation provides.
- Combining manual and automatic instrumentation is often recommended for comprehensive coverage. For example, OTel supports hybrid approaches where autoinstrumentation captures baseline telemetry and manual instrumentation adds business-critical context.
- AI-assisted instrumentation and instrumentation-as-code approaches simplify manual instrumentation and accelerate deployment in complex service environments.

Related Architecture Research

Assessing OpenTelemetry for Enterprise Observability

Monitoring and Observability for Infrastructure and Applications

Reference Architecture Brief: Best Practice Framework for Observability and Incident Response

Reference Architecture Brief: Kubernetes Infrastructure

Solution Path for Optimizing Monitoring and Observability

This research note is restricted to the personal use of .

Code-Level Instrumentation in Service-Level Observability Considerations

When implementing code-level instrumentation in service-level observability, consider the following challenges and possible pitfalls:

- Use OTel as the standard framework across SDKs, libraries, and agents to prevent vendor lock-in and maintain consistent telemetry formats.
- Validate the role of any Kubernetes operator provided by an observability platform because operators may perform tasks beyond auto-instrumentation, such as deploying collectors or managing configuration.
- Plan for fleet management at scale by using centralized policies and orchestration to avoid version drift and configuration gaps across services.
- Tag spans and metrics with business context, such as customer tier or transaction type, during instrumentation to enable correlation with business telemetry collected elsewhere.

System-Level Instrumentation

Back to top

Technologies that generate telemetry at this level operate within the OS or kernel, providing deep visibility into system processes, resource consumption, and system calls. These mechanisms can be deployed natively or dynamically, often without requiring changes to the kernel source code. Modern approaches such as eBPF extend beyond dynamic tracing and profiling to observe application-layer transactions (e.g., HTTP/gRPC) and capture request metadata at runtime—without code changes. Coverage varies by protocol, kernel capabilities, and deployment model.

- **eBPF Probes —** Kernel-resident programs that enable secure, non-intrusive, high-performance telemetry collection. These probes support dynamic instrumentation of kernel and user-space events without modifying kernel source code. They can observe L7 transactions (e.g., HTTP/gRPC) via socket-level inspection or user-space hooks and emit metrics.
- **Kernel modules —** Loadable components that extend kernel functionality to emit telemetry. While historically used for system monitoring, kernel modules are increasingly avoided in favor of safer, more flexible eBPF-based approaches.

This research note is restricted to the personal use of .

- **Profilers** — Techniques for measuring resource consumption (e.g., CPU, memory, I/O) over time. Profiling can be implemented using eBPF, native kernel features like perf, or user-space tools, supporting performance analysis and anomaly detection.

Both open-source frameworks and proprietary technologies support these mechanisms. eBPF-based instrumentation and profiling are preferred for modern system-level telemetry due to their minimal overhead, dynamic deployment capabilities, and robust security. While dynamic tracing (e.g., kprobes and uprobes) captures low-level system and application events, distributed tracing is typically used to follow request flows across services, offering broader context for troubleshooting and optimization.

Example Technologies

The following are example technologies that enable system-level instrumentation:

- **eBPF Probes:** BCC (BPF Compiler Collection), bpftrace, Grafana Beyla, OpenTelemetry eBPF Instrumentation (OBI), Pixie
- **Kernel Modules:** ftrace, LTTng (Linux Trace Toolkit Next Generation), SystemTap
- **Profilers:** Elastic Universal Profiling, Grafana Pyroscope, OTel eBPF Profiler, Parca Agent, perf

This research note is restricted to the personal use of .

Key Characteristics

- Kernel-level telemetry via eBPF-powered dynamic probes captures system calls, process activity, and resource utilization, providing deep visibility into system behavior.
- Continuous profiling using eBPF or perf collects CPU, memory, and I/O metrics with minimal performance impact, expanding observability beyond traditional metrics, logs, and traces.
- Runtime probe attachment without recompiling, enabled by eBPF, reduces operational complexity compared to legacy tracing approaches.
- Fine-grained telemetry from system and application events is captured through eBPF probes or kprobes/uprobes to support root-cause analysis and correlation with higher-level signals.
- Verified execution in a constrained kernel environment is unique to eBPF, preventing unsafe operations and mitigating crashes or privilege escalation risks.
- No-code distributed tracing for L7 workloads (HTTP/gRPC) is primarily delivered by eBPF implementations that observe transactions and propagate trace context.
- Local execution insights from kernel-level spans complement code-level tracing, though eBPF-derived spans often lack business context without correlation.
- Linux-first design with emerging Windows support reflects eBPF's maturity and the trend toward broader OS-level telemetry standardization.
- Open-source and commercial observability platforms integrate eBPF alongside alternatives like perf and SystemTap, enabling flexible deployment strategies.

Related Architecture Research

Assessing OpenTelemetry for Enterprise Observability

Hype Cycle for Monitoring and Observability, 2025

Monitoring and Observability for Infrastructure and Applications

Reference Architecture Brief: Best Practice Framework for Observability and Incident Response

Solution Path for Optimizing Monitoring and Observability

This research note is restricted to the personal use of .

System-Level Instrumentation in Service-Level Observability Considerations

When implementing system-level instrumentation in service-level observability, consider the following challenges and possible pitfalls:

- Validate the maturity of distributed tracing capabilities in eBPF frameworks because these features vary by protocol (e.g., HTTP/gRPC) and may require fallback to code-level instrumentation for full coverage.
- Anticipate operating system gaps by designing for uneven feature availability across Linux and Windows platforms.
- Integrate diagnostic tools with the telemetry pipeline since some eBPF utilities provide local troubleshooting data but do not export telemetry to back-end systems.
- Govern resource usage for continuous profiling in high-density environments to prevent performance overhead and resource contention.
- Confirm the security posture for kernel-level instrumentation by ensuring privilege models and isolation mechanisms align with compliance requirements.

Network-Level Instrumentation

Back to top

Network-level instrumentation captures service-level telemetry by observing network traffic and service interactions, rather than instrumenting application code or kernel functions. This method provides external, language-agnostic visibility into service-to-service communication, request behavior, and network-layer performance. It is especially valuable in distributed systems where modifying application code is impractical or where heterogeneous environments require a consistent observability layer. Common instrumentation mechanisms include:

- **Proxies:** Intercept and observe traffic between services, enabling visibility into request paths, latency, and errors without modifying application code.
- **Sidecars:** Colocated components that collect, forward, or enrich telemetry data alongside application containers, often used to decouple observability logic from business logic.

This research note is restricted to the personal use of .

- **Service Mesh Components:** Deploy proxies and sidecars as part of a broader observability and traffic control strategy, supporting telemetry collection, policy enforcement, and routing.
- **eBPF Probes:** Attach to kernel-level networking functions to capture packet and flow data without modifying application code, enabling low-overhead network observability.

These technologies operate at the communication layer and provide a unified view of network behavior, enabling monitoring, policy enforcement, and traffic shaping alongside observability. They are a critical complement to code-level and system-level instrumentation.

In addition to active instrumentation, infrastructure components often emit passive telemetry such as flow logs and traffic metadata. These sources provide visibility into traffic that may be dropped or filtered before reaching application-level instrumentation, such as denied requests blocked by firewalls or security groups.

Examples include:

- **VPC Flow Logs:** Cloud-native logging of accepted/rejected traffic at the subnet or interface level.
- **NetFlow/sFlow/IPFIX:** Standards for collecting IP traffic information from routers and switches in hybrid or on-premises environments.

These telemetry sources are essential for understanding network behavior across cloud and hybrid environments, especially for security, compliance, and troubleshooting scenarios.

Example Technologies

The following are technology examples that enable network-level instrumentation:

- **Proxies:** Envoy Proxy (Envoy), Linkerd Proxy (Linkerd), HAProxy (HAProxy Technologies)
- **Sidecars:** Collector Sidecar (OpenTelemetry), Log Forwarder Sidecar (Fluent Bit)
- **Service Mesh Components:** Telemetry Module (Istio), Telemetry Add-on (Linkerd), Observability Extension (Kuma)

This research note is restricted to the personal use of .

- **eBPF Probes:** Network Flow Probe (Cilium), Flow Observability Probe (Hubble), Telemetry Probe (Pixie)
- **Infrastructure-Generated Telemetry:** VPC Flow Logs (AWS/GCP/Azure), NetFlow (Cisco), sFlow (InMon), IPFIX (IETF Standard), Deep Observability Pipeline (Gigamon)

Key Characteristics

- Captures telemetry from service-to-service communication without requiring changes to application code, enabling observability in polyglot and third-party environments.
- Provides visibility across OSI Layers 3 through 7, including IP routing, TCP behavior, TLS handshakes, and application-layer protocols such as HTTP and gRPC.
- Observes and validates trace context propagation between services, helping ensure continuity and correctness in distributed tracing implementations.
- Offers an external perspective on service behavior, complementing the internal view of code-level instrumentation and the low-level insights of system-level instrumentation.
- Delivers consistent telemetry across services regardless of language or framework, reducing variability and improving correlation.
- Generates telemetry based on actual network traffic using proxies, sidecars, or eBPF-based flow tracing, independent of application logic or runtime.
- Enriches telemetry with metadata such as service identity, request path, and protocol details, supporting correlation across telemetry types and layers.
- Enables topology-aware observability by mapping service interactions and dependencies through observed traffic flows.
- Scales efficiently in dynamic environments by leveraging mechanisms like eBPF for low-overhead, kernel-resident telemetry collection.

Related Architecture Research

Assessing OpenTelemetry for Enterprise Observability

Network Monitoring and Analytics for Data-Driven Observability

This research note is restricted to the personal use of .

Network-Level Instrumentation in Service-Level Observability Considerations

When implementing network-level instrumentation in service-level observability, consider the following challenges and possible pitfalls:

- Ensure trace context continuity by validating proxy and service mesh configurations because misconfigurations can break distributed traces and lead to incomplete observability.
- Standardize telemetry enrichment at the network layer to avoid missing or mismatched metadata such as service identity, route, or protocol details, which reduces correlation accuracy.
- Plan for deployment complexity when rolling out sidecars or DaemonSets at scale, especially in dynamic environments with frequent service churn.
- Monitor performance overhead introduced by proxies and sidecars since excessive latency or resource contention can impact service health.
- Confirm kernel-level dependencies for eBPF-based telemetry because OS version and privilege requirements may limit adoption in heterogeneous environments.
- Address security and compliance risks when exposing traffic metadata for observability by implementing strict controls to prevent data leakage.
- Normalize telemetry across hybrid and multicloud networking stacks to mitigate variability in protocols and overlay networks that complicate correlation.
- Align network telemetry aggregation with failure domains such as node, zone, or cluster boundaries to improve impact analysis and reduce misinterpretation.
- Manage operational overhead for service mesh policies and certificates carefully because misconfiguration during scale-out or upgrades can increase complexity and risk.
- Plan for encrypted traffic visibility as eBPF-based approaches cannot inspect TLS payloads without additional configuration or proxies for termination, which may increase complexity and risk.

This research note is restricted to the personal use of .

Infrastructure Telemetry

Back to top

Infrastructure telemetry captures metrics, logs, events, and even profiling data from the hosts, containers, clusters, platform services, and cloud provider telemetry sources that support application workloads. It aims to reveal how resource utilization, component health, orchestration actions, and cloud-service behavior affect service-level outcomes. This telemetry is enriched with standardized metadata (e.g., service.name, namespace, region, trace context) to support correlation, filtering, and attribution.

Common technologies include:

- **Telemetry Agents:** Host-resident components that collect telemetry, buffer data locally during network disruptions, and transmit it to collection endpoints.
- **Exporters:** Interfaces that expose telemetry from services or systems (e.g., databases, message queues, cloud APIs) for collection via pull or push mechanisms by collection endpoints (e.g., Prometheus or OTLP collectors).
- **Forwarders:** Edge components that aggregate telemetry from multiple sources, apply lightweight transformation or buffering, and relay data to collection endpoints. Often called shippers or relays, they operate close to the source and are distinct from centralized telemetry pipelines.
- **Operators:** Kubernetes-native controllers that deploy and manage telemetry components; some support eBPF-based visibility for low-overhead kernel and application-layer telemetry (Linux-first, with limited Windows support).

This architectural approach differs from traditional infrastructure monitoring by attributing telemetry to defined service boundaries and dependency paths. It enables consistent coverage across hybrid and multicloud environments, supports dynamic kernel-level visibility, and accommodates overlapping roles among telemetry agents, exporters, and forwarders based on deployment context. Infrastructure telemetry lays the foundation for downstream correlation by surfacing resource conditions alongside code, system, and network-level signals.

Example Technologies

The following are example technologies that emit infrastructure telemetry:

This research note is restricted to the personal use of .

- **Telemetry Agents:** collectd, Datadog Agent, MetricsHub Agent, Netdata Agent, OpenTelemetry Collector
- **Exporters:** AWS CloudWatch Exporter, Prometheus Exporter, OpenTelemetry Exporters (OTLP, Jaeger, Zipkin)
- **Forwarders:** Fluent Bit, Grafana Loki Promtail, Splunk Universal Forwarder, Vector
- **Operators:** Prometheus Operator, OpenTelemetry Operator, New Relic Kubernetes Operator, Inspektor Gadget, Cilium Operator

Key Characteristics

- Attributes infrastructure signals to defined service boundaries and dependency paths, enabling correlation of resource pressure or failures to specific workloads.
- Maintains consistent telemetry coverage across hybrid and multicloud environments using telemetry agents, exporters, and forwarders deployed across hosts, containers, and cloud services. In this context, telemetry agents are software components that run alongside applications to inject telemetry logic at runtime and are not autonomous or decision-making AI systems.
- Enables dynamic, low-overhead kernel-level visibility via eBPF probes (Linux-first), capturing system calls, process life cycles, and network behavior without code changes or kernel recompilation.
- Buffers telemetry at the edge to preserve continuity during collector outages or network partitions, reducing observability gaps in distributed environments.
- Supports metadata enrichment for correlation and filtering, using standardized attributes such as service.name, namespace, region, and trace context.
- Accommodates overlapping roles of telemetry agents, exporters, and forwarders by emphasizing deployment context (host-resident, pull-based, edge relay) rather than rigid functional classification.
- Integrates Kubernetes platform telemetry via operators, telemetry agents, or eBPF-enhanced controllers, supporting flexible privilege models and performance requirements.

Related Architecture Research

Assessing OpenTelemetry for Enterprise Observability

This research note is restricted to the personal use of .

Market Guide for Infrastructure Monitoring Tools

Monitoring and Observability for Infrastructure and Applications

Reference Architecture Brief: Best Practice Framework for Observability and Incident Response

Reference Architecture Brief: Kubernetes Infrastructure

Infrastructure Telemetry in Service-Level Observability Considerations

When implementing infrastructure telemetry in service-level observability, consider the following challenges and possible pitfalls:

- Map telemetry to service boundaries by consistently tagging metadata such as pod labels or cloud resource tags so that infrastructure signals align with service definitions.
- Normalize telemetry across hybrid and multicloud environments because variability in APIs, formats, and CNI implementations can complicate correlation and analysis.
- Configure edge buffering in telemetry agents or forwarders correctly to prevent data loss or backpressure during extended collector outages or network partitions.
- Validate kernel-level visibility features such as eBPF support since incomplete or inconsistent implementations can create gaps in telemetry or require fallback mechanisms.
- Balance resolution for high-frequency metrics and profiling data against storage and compute costs to avoid excessive overhead in large-scale environments.
- Confirm that Kubernetes operators managing telemetry components do not conflict with other controllers to prevent resource contention or configuration drift.
- Optimize forwarding configurations, including batch size, retry policies, and compression, because misconfiguration can cause delivery delays or unexpected egress charges.
- Address security and compliance constraints on infrastructure metadata, including cloud resource tags and system-level logs, to prevent inadvertent exposure.

This research note is restricted to the personal use of .

Telemetry Management

Back to top

Telemetry management is the architectural capability responsible for orchestrating telemetry — logs, metrics, traces, events, and profiles — throughout its life cycle, from initial ingestion to delivery at the observability platform. It ensures telemetry is optimized to enhance service-level observability, regardless of source or deployment topology.

To manage telemetry effectively, organizations must implement five core sub-capabilities:

- **Collect:** Ingest telemetry from source systems (e.g., applications, infrastructure, containers) via telemetry agents, exporters, or direct integrations.
- **Transform:** Normalize and reformat telemetry into standardized schemas and protocols (e.g., OTLP, Prometheus), applying parsing, masking, and compliance rules.
- **Enrich:** Attach contextual metadata (e.g., service identifiers, environment, region, trace context) to increase analytical value and support downstream correlation.
- **Reduce:** Filter, deduplicate, sample, or aggregate telemetry at the collector or pipeline stage to control data volume, cost, and noise while preserving critical diagnostic fidelity.
- **Route:** Policy-driven delivery of telemetry streams to one or more observability platforms, data lakes, or archives based on business rules and operational logic.

To implement these sub-capabilities, organizations rely on two complementary technology categories that work together to form a unified telemetry management architecture:

- **Telemetry collectors:** First-mile telemetry agents or services deployed close to the source (hosts, containers, applications) to acquire telemetry. While their primary role is data collection, some collectors support lightweight transformation, enrichment, or reduction before forwarding data onward.
- **Telemetry pipelines:** Centralized or distributed frameworks that receive telemetry from multiple collectors, then apply advanced processing — transforming, enriching, reducing, and routing data to optimize flow and governance.

This research note is restricted to the personal use of .

Collectors and pipelines form a unified telemetry management architecture — collectors gather data at the edge, while pipelines optimize, govern, and deliver telemetry to its destinations.

Example Technologies

The following are example technologies that enable telemetry management:

Telemetry Collectors

- **Open Source:** Fluent Bit, Grafana Alloy, OTel Collector, Prometheus Server, Telegraf
- **Proprietary:** Datadog Agent, Dynatrace OneAgent, Splunk Universal Forwarder

Telemetry Pipelines

- **Open Source:** BindPlane OP, Datadog Vector
- **Proprietary**: Cribl Stream, Datadog Observability Pipelines, Edge Delta, Mezmo Telemetry Pipeline

This research note is restricted to the personal use of .

Key Characteristics

- Telemetry management acts as the architectural backbone for observability data flow, ensuring that collection, processing, and delivery occur in a unified, policy-driven manner across distributed environments.
- Collectors positioned close to the source minimize latency and preserve fidelity, while the pipeline centralizes advanced processing to enforce normalization, enrichment, and routing policies.
- The architecture leverages modularity by separating first-mile collection from centralized governance, enabling independent scaling and evolution of telemetry sources and destinations.
- Enrichment within the pipeline introduces shared context — such as trace IDs and service metadata — that supports correlation across logs, metrics, traces, and profiles for service-level observability.
- Reduction techniques applied at multiple stages optimize cost and performance without sacrificing high-value telemetry, balancing ingestion volume with analytical requirements.
- Routing logic embedded in the pipeline aligns telemetry delivery with architectural intent, directing data to the observability platform, data stores, or other analytics engines based on operational policies.
- Telemetry management provides resilience through buffering, retry mechanisms, and dynamic routing, ensuring continuity of observability even during network disruptions or platform outages.

Related Architecture Research

Guidance Framework for Deploying Centralized Log Management and Monitoring

Reference Architecture Brief: Best Practice Framework for Observability and Incident Response

Telemetry Management in Service-Level Observability Considerations

When implementing telemetry management in service-level observability, consider the following challenges and possible pitfalls:

- Deploy collectors close to telemetry sources to reduce latency and preserve fidelity, while planning for the operational overhead introduced in distributed environments.

This research note is restricted to the personal use of .

- Consolidate multisignal collectors to minimize agent sprawl and simplify integration across heterogeneous sources and formats.
- Utilize telemetry pipelines for policy-driven transformation, enrichment, reduction, and routing, while monitoring for latency and resource consumption, especially when applying complex transformations (e.g., masking, enrichment) in high-volume environments.
- Configure buffering, retry, and backpressure mechanisms in the pipeline carefully to maintain delivery resilience without creating bottlenecks.
- Enrich telemetry with service metadata and trace context to improve correlation across logs, metrics, traces, and profiles, while managing cardinality to avoid excessive storage costs.
- Apply pipeline-level reduction (sampling, deduplication, aggregation) with bias checks (e.g., tail-based capture of error/outlier traces) to control cost without undermining rare-event diagnostics.

Telemetry Retention

Back to top

Telemetry retention is the capability that stores, manages, and retrieves observability data — metrics, logs, traces, events, and profiles — across multiple storage tiers. These tiers are designed to balance speed, cost, and durability:

- Hot storage supports low-latency queries and short-term retention for real-time troubleshooting.
- Cold storage offers cost-efficient retention for data that remains searchable but is accessed less frequently.
- Frozen storage provides long-term archival for compliance or forensic needs, often requiring rehydration before analysis.

The architecture supports signal-aware strategies, techniques that reduce telemetry volume or enhance its utility based on signal type. These strategies are not performed by retention technologies themselves, but are typically applied at different stages of the telemetry life cycle:

- **Metrics** are usually downsampled or rolled up by observability platforms before entering long-term storage, preserving trends while reducing volume.

This research note is restricted to the personal use of .

- **Traces** are often sampled at telemetry collection or within the pipeline using head or tail-based logic to retain spans with errors or latency outliers.
- **Logs** and **events** are typically compressed, deduplicated, or tiered within the pipeline, optimizing cost and performance before storage.
- **Profiles** are generally aggregated or retained at a lower frequency by observability platforms or telemetry agents to support long-term performance and capacity planning.

These strategies are enforced through the telemetry pipeline and observability platform, which handle routing, encryption, and metadata enrichment at ingest. Because telemetry is tagged by service context, life cycle rules, such as aging data out of hot tiers, transitioning it to colder ones, or applying legal holds, should be defined and executed in alignment with service boundaries. This ensures that historical queries, rehydration, and correlation remain accurate across services, environments, and trace identifiers.

Transient buffering layers are often used earlier in the telemetry pipeline to absorb backpressure and decouple ingestion from storage, ensuring reliable delivery at scale. Profiling telemetry is typically retained in specialized formats optimized for resource usage analysis. Recent data is stored in embedded platforms for interactive troubleshooting, while older profiles are often tiered to object storage to support long-term performance and capacity planning.

To support these retention strategies, organizations rely on a mix of specialized technologies:

- **Embedded platform store:** Provides short-term, high-speed storage for interactive troubleshooting, often built into observability platforms or telemetry tools. May be optimized for specific telemetry types such as metrics, traces, logs, or profiling data.
- **Time-series metrics store:** Scales to handle millions of time-stamped datapoints per minute, enabling rapid aggregation and visualization.
- **Real-time analytics store:** Supports sub-second analysis of large event streams or traffic spikes in latency-sensitive scenarios.
- **Log indexing store:** Enables scalable search and filtering of service-tagged logs for troubleshooting and compliance.

This research note is restricted to the personal use of .

- **Long-term metrics store:** Stores downsampled metrics for multiyear trend analysis and SLA audits.
- **Object storage:** Elastic, low-cost storage for logs, traces, and metrics artifacts with preserved metadata for rehydration.
- **Frozen (archival) storage:** Lowest-cost tier for telemetry under legal or compliance hold, accessed via controlled rehydration.

Example Technologies

The following are example technologies that enable telemetry retention:

- **Embedded platform store:** Datadog, Dynatrace Grail, Grafana Pyroscope, New Relic NRDB, Parca, Splunk Platform, Sumo Logic
- **Time-series metrics store:** Amazon Timestream, Grafana Mimir, InfluxDB, TimescaleDB, VictoriaMetrics
- **Real-time analytics store:** Apache Druid, Apache Pinot, ClickHouse, Rockset, StarRocks
- **Log Indexing Store:** Elasticsearch, Grafana Loki, OpenSearch, Splunk Enterprise, Falcon LogScale (Humio)
- **Long-term metrics store:** Cortex, Grafana Mimir, M3, Prometheus + Thanos, VictoriaMetrics
- **Object storage:** Amazon S3, Azure Blob Storage, Google Cloud Storage, MinIO, Oracle Cloud Object Storage
- **Frozen (archival) storage:** Amazon S3 Glacier, Azure Archive Storage, Google Cloud Archive, IBM Cloud Archive, Wasabi

This research note is restricted to the personal use of .

Key Characteristics

- Integrates with telemetry pipelines and observability platforms to ensure data normalization, enrichment, and routing align with observability and compliance policies.
- Supports multiple deployment models and can function as stand-alone storage, embedded components of broader data platforms, or integrated features within observability solutions.
- Enables telemetry forwarding to external storage systems connected to observability platforms for ingestion, enrichment, and analysis.
- Optimizes short-term retention through embedded platform storage, while long-term retention typically relies on decoupled systems with life cycle automation and compliance controls.
- Implements policy-driven tiering and life cycle automation to manage data across hot, cold, and frozen tiers for cost and performance efficiency.
- Applies signal-aware retention strategies for metrics, logs, events, traces, and profiles to preserve analytical value while controlling data volume.
- Accounts for diverse telemetry types and access patterns, ensuring historical data remains queryable and useful for analysis.
- Supports federated query engines or virtualized access layers that enable cross-store queries without physical data movement, reducing duplication and improving flexibility.
- Maintains service-level context through metadata tagging across all tiers to support accurate correlation during historical analysis.
- Account for rehydration complexity and cost when using frozen storage tiers for compliance or forensic needs, as these tiers are not designed for immediate query access.

Related Architecture Research

Reference Architecture Brief: Best Practice Framework for Observability and Incident Response

Top Trends in Enterprise Data Storage for 2025

This research note is restricted to the personal use of .

Telemetry Retention in Software-Level Observability Considerations

When implementing telemetry retention in service-level observability, consider the following challenges and possible pitfalls:

- Evaluate whether independent storage is required for compliance, audit, or portability needs because this may involve using object storage or log indexing stores for long-term access.
- Define retention policies that balance cost and performance by limiting observability platforms to short-term retention and using tiered storage for archival data.
- Match storage tiers to access patterns by keeping recent, high-frequency telemetry in hot tiers for fast queries and moving older data to cold or frozen tiers for cost efficiency.
- Confirm regulatory and data sovereignty requirements before selecting storage solutions to ensure geographic control over data residency.

Telemetry Analysis

Back to top

Telemetry analysis is the architectural capability that analyzes processed and retained observability data, including metrics, logs, traces, events, and profiles, to deliver actionable insights for managing service level objectives (SLOs) and ensure service health. This capability transforms enriched telemetry into outcomes that accelerate detection, diagnosis, and prediction of service-level conditions. It enables human-led investigations and automated workflows, ensuring insights stay anchored to service boundaries rather than individual infrastructure components.

Telemetry analysis builds on upstream telemetry management and retention, applying advanced techniques, including predictive, causal, and generative AI, to correlate signals, identify anomalies, and forecast potential risks. To ensure transparency, outputs are linked to supporting evidence and used to inform dashboards, reports, and system-facing integrations such as event intelligence and ITSM platforms. These insights accelerate incident resolution, support proactive SLO compliance, and improve reliability across modern and traditional architectures.

To deliver these outcomes, Telemetry analysis combines multiple analytical capabilities that work together to interpret service behavior and drive action:

This research note is restricted to the personal use of .

- **Query engine:** Provides an intuitive interface for executing ad hoc queries and exploring service-level telemetry. It helps teams validate hypotheses, troubleshoot issues, and create custom visualizations for deeper operational insight.
- **Telemetry correlation:** Automatically discovers relationships among metrics, logs, traces, events, and profiles by using shared metadata such as service identifiers, environment tags, and trace context. This links symptoms to contributing factors across multiple layers. Profiles add resource behavior details (e.g., CPU or memory trends), while events provide operational context (e.g., deployments or scaling actions), creating richer causal chains and more accurate impact analysis.
- **Predictive AI:** Applies machine learning models to historical telemetry to forecast service behavior. Common use cases include predicting resource saturation, anticipating performance degradation, and providing early warnings for potential breaches of service health thresholds.
- **Causal AI:** Determines root causes by modeling cause-and-effect relationships across services and infrastructure. This subcomponent isolates the origin of incidents, reducing mean time to resolution by focusing remediation on the true source.
- **Generative AI:** Enables natural language interaction and automated summarization of observability data. Capabilities include conversational queries, incident report generation, and AI-assisted troubleshooting recommendations, making insights accessible to a broader range of stakeholders.

Example Technologies

The following are example technologies that enable telemetry analysis:

- **Query engine:** AWS (Amazon OpenSearch Service), Elastic (Search AI Platform), Microsoft Azure (Kusto Query Language/KQL), Prometheus (PromQL), Splunk (Search Processing Language/SPL)
- **Telemetry correlation:** BMC Helix (Discovery), Datadog (Unified Platform), Dynatrace (Smartscape and PurePath), LogicMonitor (Edwin AI for event correlation), New Relic (Applied Intelligence), Splunk (IT Service Intelligence/ITSI)
- **Predictive AI:** Datadog (SLO management and alerting), Dynatrace (Davis predictive modeling), Elastic (Machine Learning), ScienceLogic (Skylar AI), SolarWinds (Anomaly Detection)

This research note is restricted to the personal use of .

- **Causal AI:** AWS (CloudWatch Investigations), Chronosphere (Differential Diagnosis/DDx), Dynatrace (Davis automated root cause analysis), IBM (Instana), LogicMonitor (GenAI-powered RCA), New Relic (AI-assisted troubleshooting), Virtana (Causal Analysis)
- **Generative AI:** AWS (Natural Language Query), BMC (HelixGPT), Coralogix (Cora), Elastic (AI Assistant), LogicMonitor (LM Co-Pilot), Microsoft (Copilot in Azure), New Relic (Generative AI Interface), Oracle (GenAI-powered Observability Assistant), Splunk (Cisco AI Assistant), Sumo Logic (Mo Copilot)

Key Characteristics

- Anchors analytics to clearly defined service boundaries so insights reflect service health, not just infrastructure metrics.
- Leverages telemetry that is normalized and enriched with service identifiers and trace context, enabling correlation across signals in a single analytical view.
- Uses dynamic service maps and dependency relationships to interpret issues in context, supporting accurate fault isolation across distributed workflows.
- Incorporates operational events (such as deployments or scaling actions) and resource profiles (like CPU or memory trends) to strengthen correlation and improve causal accuracy.
- Coordinates predictive, causal, and generative analytics within the service boundary to detect emerging risks, pinpoint root causes, and explain findings in service terms.
- Produces evidence-linked outputs, such as related spans, log clusters, and profile comparison, to make conclusions transparent and auditable.
- Supports calculation of service-level indicators (SLIs) and proactive management of service-level objectives (SLOs) by aligning analytics to service health targets and reliability thresholds.

Related Architecture Research

Assessing OpenTelemetry for Enterprise Observability

Enhance ITSM Practices With Observability Platforms

Monitoring and Observability for Infrastructure and Applications

This research note is restricted to the personal use of .

Reference Architecture Brief: Best Practice Framework for Observability and Incident Response

Solution Path for Optimizing Monitoring and Observability

The State of AI in Observability

Telemetry Analysis in Service-Level Observability Considerations

When implementing telemetry analysis in service-level observability, consider the following challenges and possible pitfalls:

- Ensure consistent service attribution across all telemetry signals because missing or inconsistent identifiers will undermine correlation and causal analysis.
- Maintain accurate and up-to-date service maps since incomplete or stale topology data can lead to misleading conclusions during root cause analysis.
- Capture profiles and operational events with service metadata to provide critical context for anomaly detection and improve causal confidence.
- Retain sufficient historical telemetry aligned to service boundaries because predictive and causal models require long-term data to forecast risks and validate reliability targets.
- Make generative AI outputs explainable and link them to supporting telemetry artifacts such as spans, logs, and profiles to maintain trust and operational transparency.
- Confirm interoperability when using mixed-vendor stacks for analytics because advanced features often rely on proprietary formats and may increase complexity or risk of lock-in.
- Validate the maturity of generative AI features in observability platforms, as adoption is uneven and capabilities may be limited compared to predictive or causal analytics.

Telemetry Visualization

Back to top

This research note is restricted to the personal use of .

Telemetry visualization converts service-level telemetry into views that make complex relationships easier to understand. It aims to help teams interpret a service's behavior across its dependencies and operational context, rather than forcing them to piece together isolated charts or log streams. By grouping telemetry under the same service boundary and layering context, such as deployment events or dependency maps, visualization enables faster troubleshooting, validation of SLO, and alignment of technical insights with business outcomes. Service-level visualizations can be categorized into three subcomponents:

- **Service views:** Graphical representations of telemetry that reveal patterns, anomalies, and trends within a defined service boundary. These include time-series charts showing a specific service's latency, heatmaps highlighting error rates across regions, and flame graphs from continuous profiling to pinpoint CPU hot spots within critical workflows.
- **Role-based dashboards:** Collections of visualizations organized around a role or operational theme. A DevOps dashboard might combine deployment timelines with real-time service health indicators, while an SRE dashboard correlates distributed traces with infrastructure telemetry to confirm SLO compliance. Reliability dashboards often map service dependencies and highlight bottlenecks across workflows, making it clear where failures propagate.
- **Service reports:** Structured outputs that summarize historical telemetry and analysis. Weekly reliability reports can show SLO attainment and recurring incident patterns, post-incident reports link root cause to impacted services, and executive summaries connect service performance trends to business KPIs for strategic planning.

Modern observability platforms extend visualization beyond static charts by introducing adaptive and AI-driven capabilities. These features personalize dashboards based on user roles, highlight anomalies automatically, and adjust views as service conditions change. The result is visualization that stays actionable and aligned with operational priorities and business objectives.

Example Technologies

The following are example technologies for telemetry visualization:

- **Service views:** Grafana, Jaeger, Kibana, Prometheus, Pyroscope

This research note is restricted to the personal use of .

- **Role-based dashboards:** Datadog, Dynatrace, Elastic Observability, New Relic, Splunk ITSI
- **Service reports:** Amazon QuickSight, Datadog Reporter, Looker, Microsoft Power BI, Tableau

Key Characteristics

- Presents telemetry in layered views that combine service health indicators with dependency relationships, enabling interpretation of cascading impacts across workflows.
- Maintains continuity of trace context and service identifiers across visual constructs, ensuring accurate correlation between charts, dashboards, and reports.
- Supports dynamic composition of dashboards from multiple telemetry sources without requiring rigid coupling to a single observability platform.
- Enables visualization of operational events, such as deployments or scaling actions, alongside service metrics and traces to provide causal context for anomalies.
- Accommodates high-cardinality service metadata without degrading dashboard performance, preserving granularity for multi-tenant or complex environments.
- Leverages adaptive visualization techniques, including AI-driven anomaly surfacing and role-aware personalization, to keep views relevant as service conditions evolve.

Related Architecture Research

Monitoring and Observability for Infrastructure and Applications

Reference Architecture Brief: Best Practice Framework for Observability and Incident Response

Solution Path for Optimizing Monitoring and Observability

The State of AI in Observability

Telemetry Visualization in Service-Level Observability Considerations

When implementing telemetry visualization in service-level observability, consider the following challenges and possible pitfalls:

This research note is restricted to the personal use of .

- Design visualization interfaces that support exploratory queries because dashboards often serve as the entry point for deeper telemetry analysis.
- Ensure dashboards reflect service health and dependencies rather than generic “single pane of glass” views to make cascading impacts clear.
- Provide historical reporting to reveal patterns and cumulative effects of outages or performance degradation over time.
- Align visualizations with operational roles such as DevOps, SRE, and business stakeholders so that dashboards remain actionable and relevant.
- Validate AI-assisted and adaptive visualization features against accurate service attribution and robust historical telemetry because incomplete data can lead to misleading recommendations.

Key Architecture Principles and Patterns

To implement service-level observability effectively, architects must adopt principles that ensure telemetry is generated, managed, and analyzed consistently across service boundaries. These principles emphasize modularity, interoperability, and actionable insights.

- **Automate instrumentation:** Reduce development overhead using auto and no-code instrumentation with open standards like OTel. This accelerates coverage and minimizes manual effort.
- **Apply system-level instrumentation:** Close observability gaps with kernel-level visibility using eBPF and continuous profiling. This provides deep diagnostics beyond application code.
- **Incorporate network-level telemetry:** Capture service-to-service communication and trace propagation via proxies, sidecars, and service mesh components. This ensures distributed workflows remain observable.
- **Collect telemetry from infrastructure:** Enrich observability with telemetry from hosts, containers, orchestration layers, and cloud platforms. This links resource behavior to service health.
- **Manage telemetry as a life cycle:** Implement collectors and pipelines to collect, transform, enrich, reduce, and route telemetry efficiently. Telemetry life cycle management ensures governance and cost control.

This research note is restricted to the personal use of .

- **Adopt standard data specifications:** Normalize telemetry using open formats (e.g., OTLP, Prometheus) to improve interoperability and reduce vendor lock-in.
- **Optimize retention and cost:** Use tiered storage and sampling strategies to balance performance, compliance, and cost across telemetry types.
- **Correlate telemetry across layers:** Enable understanding by linking traces, metrics, logs, events, and profiles. Correlation reveals dependencies and accelerates root cause analysis.
- **Visualize service health and dependencies:** Provide role-based dashboards, adaptive visualizations, and historical reporting to clarify service behavior and cascading impacts.
- **Integrate AI for analytics and automation:** Accelerate detection, prediction, and resolution with AI-driven anomaly detection, causal analysis, and automated workflows. Generative AI can simplify queries and reporting for broader accessibility.

Key Architecture Recommendations

To ensure this architecture delivers measurable reliability improvements and remains adaptable as environments evolve, apply the following recommendations:

- **Focus on connecting signals, not collecting everything:** Link logs, metrics, and traces to show service relationships and business impact.
- **Set retention rules based on service goals, not endless infrastructure storage:** Keep what supports reliability objectives.
- **Use telemetry pipelines as control points:** Normalize, enrich, and apply compliance early so data stays clean and governed.
- **Apply AI for cause and impact, not just alerts:** Help teams understand why issues happen and what they mean for the business.
- **Design dashboards for quick decisions:** Highlight cascading effects and keep views clear, not cluttered with every detail.
- **Build observability for safe rollouts:** Support canary releases and feature flags to reduce risk during deployments.
- **Plan for flexibility:** Use open standards and decoupled storage so telemetry moves easily if vendors or rules change.

This research note is restricted to the personal use of .

- **Combine technical and business context:** Link operational events with telemetry to improve root cause analysis and reliability.
- **Sample smartly:** Keep high-value traces and metrics while controlling cost and performance overhead.
- **Keep service maps accurate:** Validate dependencies often so AI insights stay trustworthy in changing environments.

Recommended by the Authors

Some documents may not be available as part of your current Gartner subscription.

Assessing OpenTelemetry for Enterprise Observability

Solution Path for Optimizing Monitoring and Observability

Monitoring and Observability for Infrastructure and Applications

Reference Architecture Brief: Best Practice Framework for Observability and Incident Response

Reference Architecture Brief: Kubernetes Infrastructure

Hype Cycle for Monitoring and Observability, 2025

Network Monitoring and Analytics for Data-Driven Observability

Enhance ITSM Practices With Observability Platforms

The State of AI in Observability

Market Guide for Infrastructure Monitoring Tools

Guidance Framework for Deploying Centralized Log Management and Monitoring

Top Trends in Enterprise Data Storage for 2025

Reference Architecture Brief: Data Observability

This research note is restricted to the personal use of .

© 2026 Gartner, Inc. and/or its affiliates. All rights reserved. Gartner is a registered trademark of Gartner, Inc. and its affiliates. This publication may not be reproduced or distributed in any form without Gartner's prior written permission. It consists of the opinions of Gartner's Business and Technology Insights Organization, which should not be construed as statements of fact. While the information contained in this publication has been obtained from sources believed to be reliable, Gartner disclaims all warranties as to the accuracy, completeness or adequacy of such information. Although Gartner insights may address legal and financial issues, Gartner does not provide legal or investment advice and its insights should not be construed or used as such. Your access and use of this publication are governed by Gartner's Usage Policy. Gartner prides itself on its reputation for independence and objectivity. Its insights is produced independently by its Business and Technology Insights Organization without input or influence from any third party. For further information, see " Guiding Principles on Independence and Objectivity." Gartner insights may not be used as input into or for the training or development of generative artificial intelligence, machine learning, algorithms, software, or related technologies.

This research note is restricted to the personal use of .